

TIMER IN ATMEGA32 – PART 2



LEARNING OBJECTIVE

- How to generate square wave of different frequency and duty cycles?
 - Control the speed of DC motor
 - Control the position of servo motor
 - Simulate an analog signal



INTRODUCTION

- In AVR Timer 1 Lecture, we learnt two features of a timer:
 - overflow interrupt
 - input capture
- Overflow interrupt
 - triggered when timer reaches its limit
 - used to measure interval that is longer than one timer cycle
 - for finding the time elapse, creating a time delay.
- Input capture
 - an interrupt triggered when there's a change in pin ICP1
 - value of Timer 1 is automatically stored in register ICR1
 - for finding period/frequency/pulse width of a signal



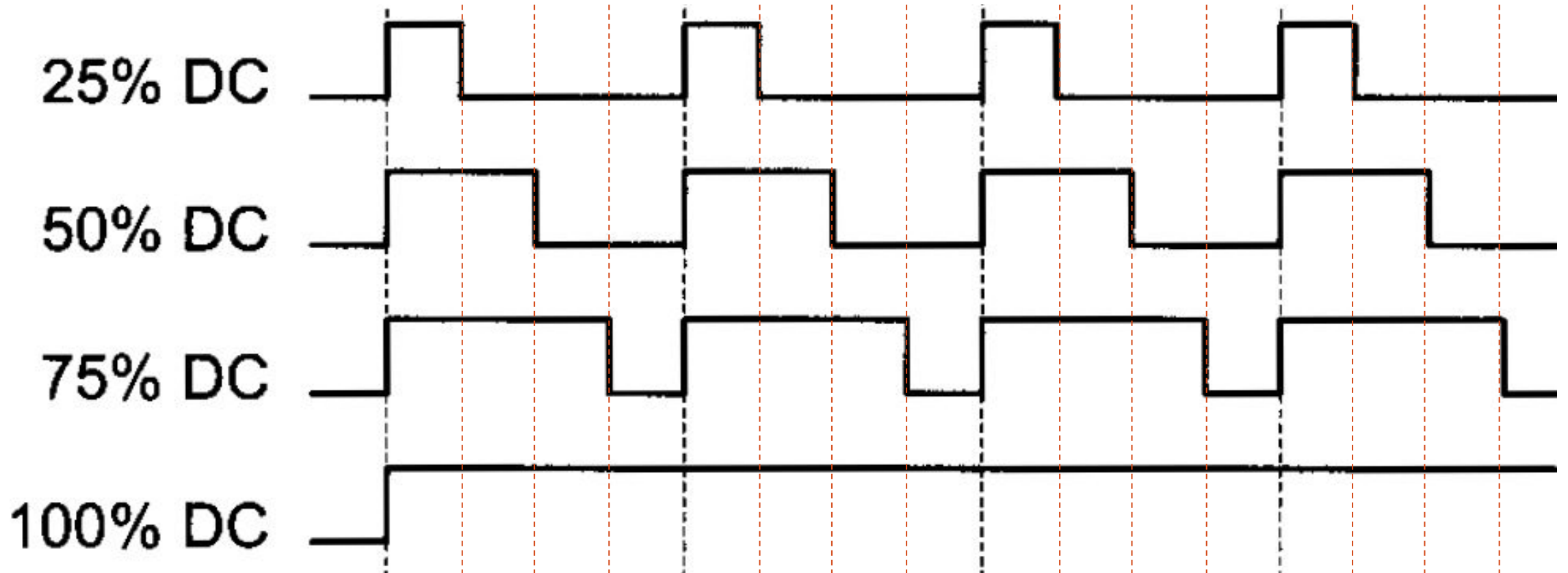
OUTPUT COMPARE

- In this lecture, we'll study another important functionality of a timer
 - output compare
- Output compare allows custom processing to be done
 - when timer reaches a preset target value
- Output compare can be used to
 - generate square waves of various shapes
 - Pulse Width Modulation
 - perform actions such as ADC at specific time instants (See lecture on ADC)
- Examples of custom processing
 - clearing timer
 - changing values of dedicated pins
 - triggering an interrupt



PULSE WIDTH MODULATION

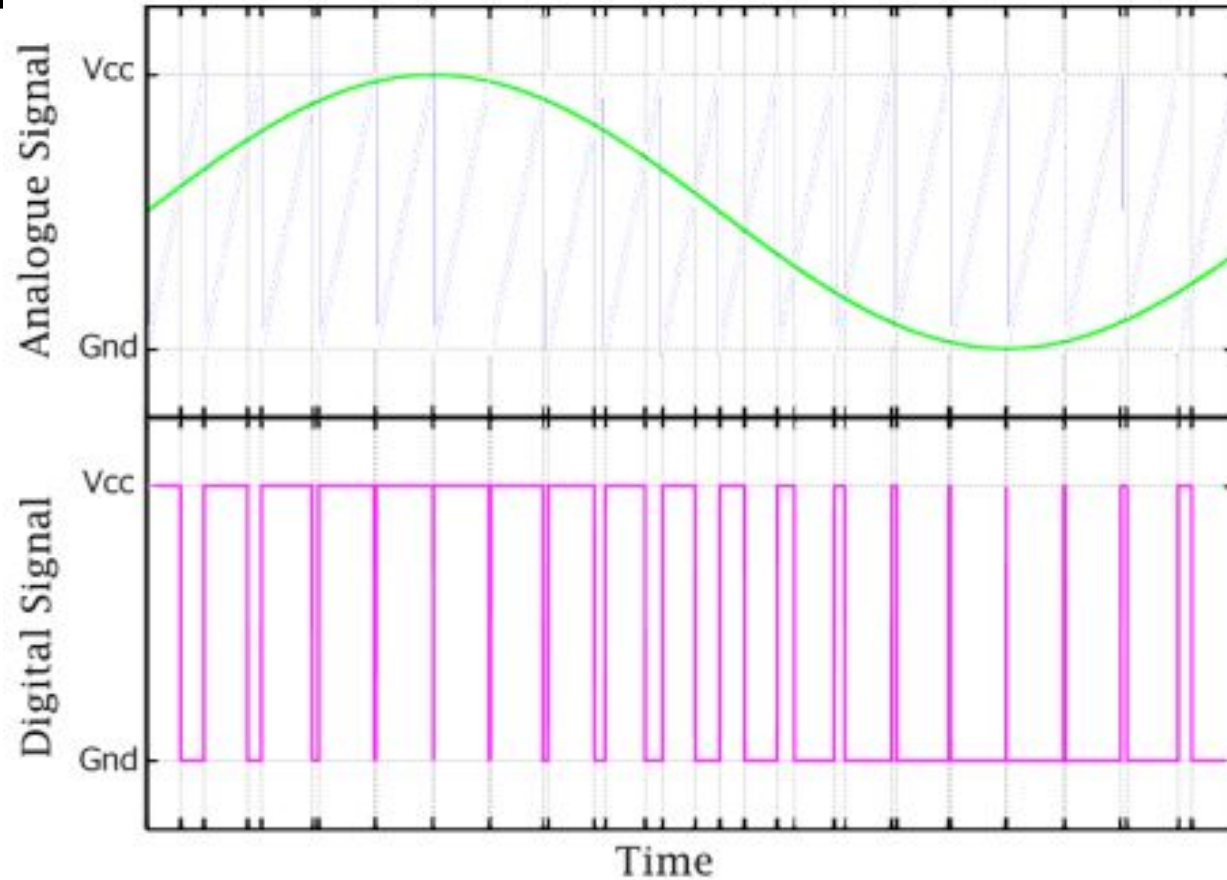
$$\text{Duty Cycle} = \frac{\text{on time}}{\text{total time}}$$



The percentage of time in which the PWM signal remains HIGH (on time) is called as duty cycle.



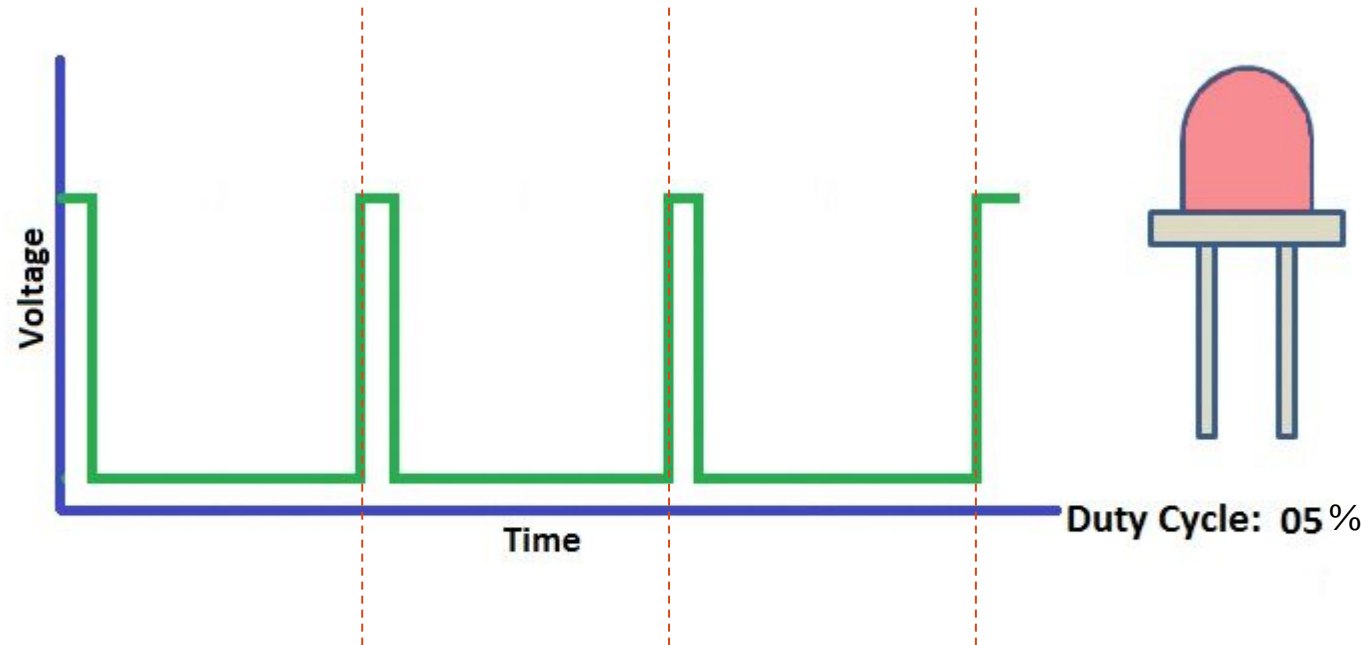
SIMULATING AN ANALOG SIGNAL



<https://www.falstad.com/circuit/e-555pulsemod.html>

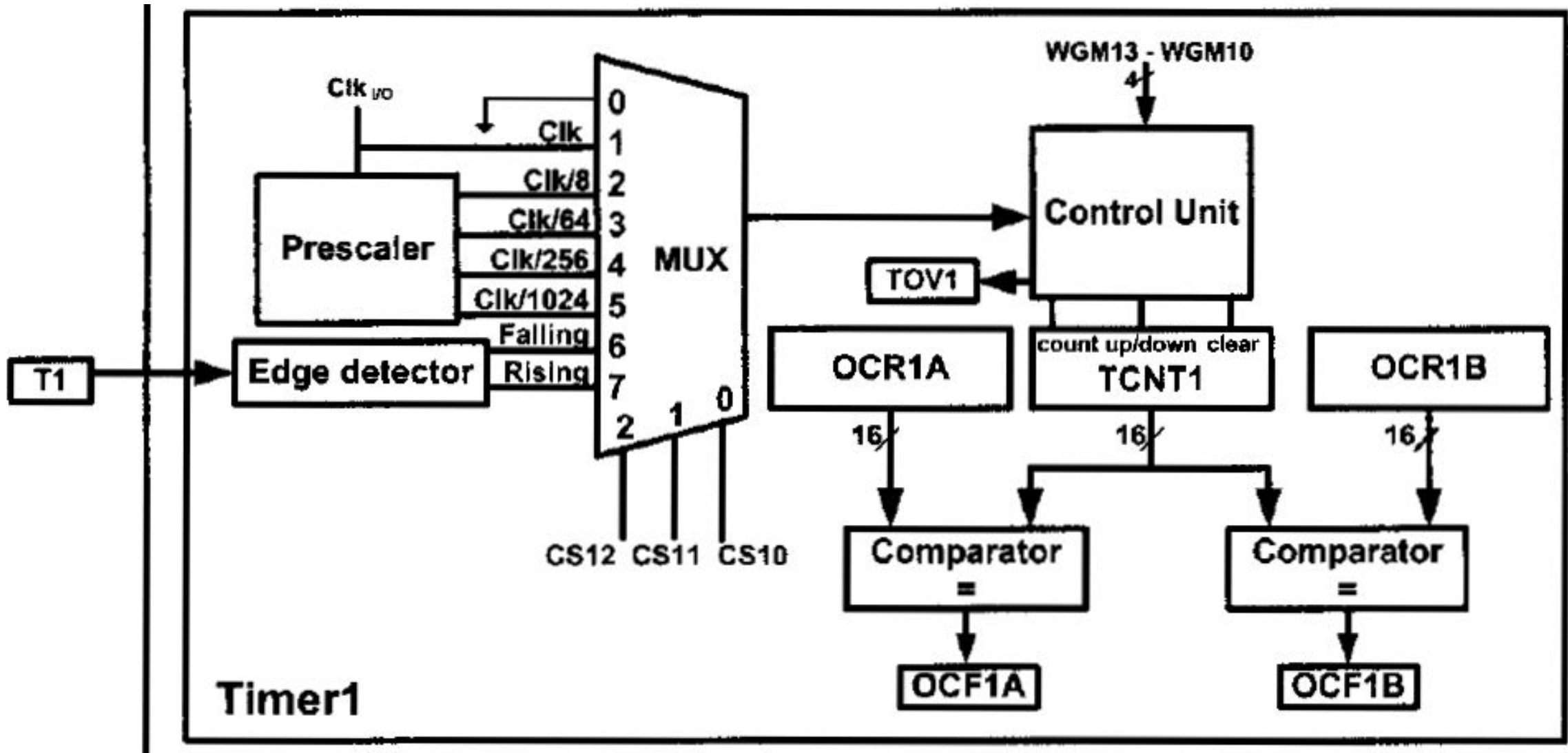


CONTROLLING POWER TO A LOAD



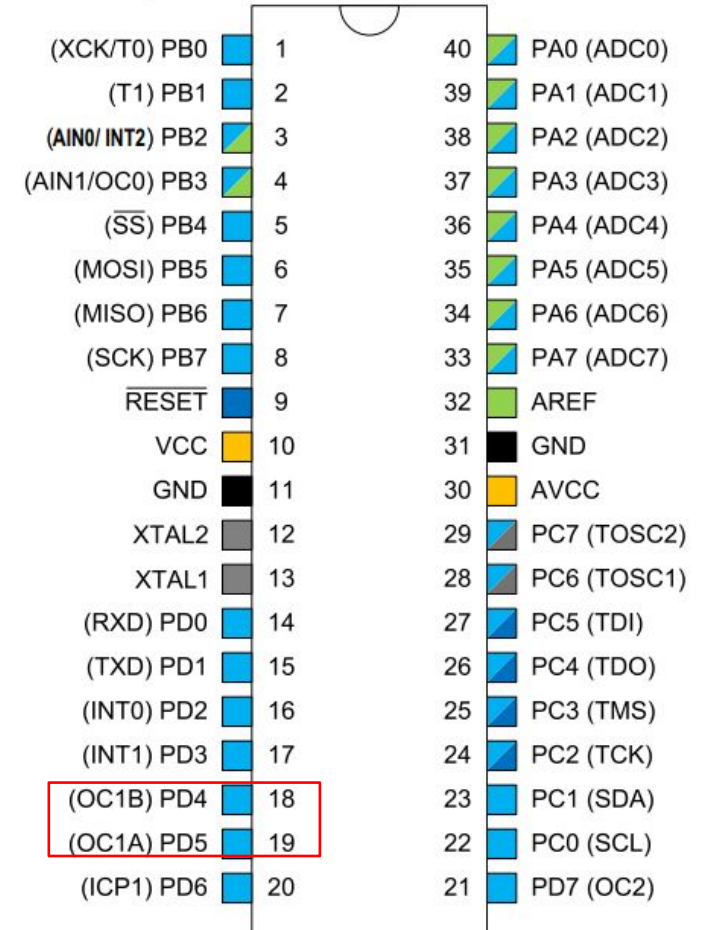
OUTPUT COMPARE - COMMON ELEMENTS

- Output compare registers (16 bit)
 - to store the target timer values
- Output compare pins
 - the values of these dedicated pins can be automatically changed (set, reset, toggled) when there is an output compare match
- Configuration registers
 - to configure the operations of timer
- Output Compare Interrupt
 - code for extra processing when there is an output compare match can be put in ISR.



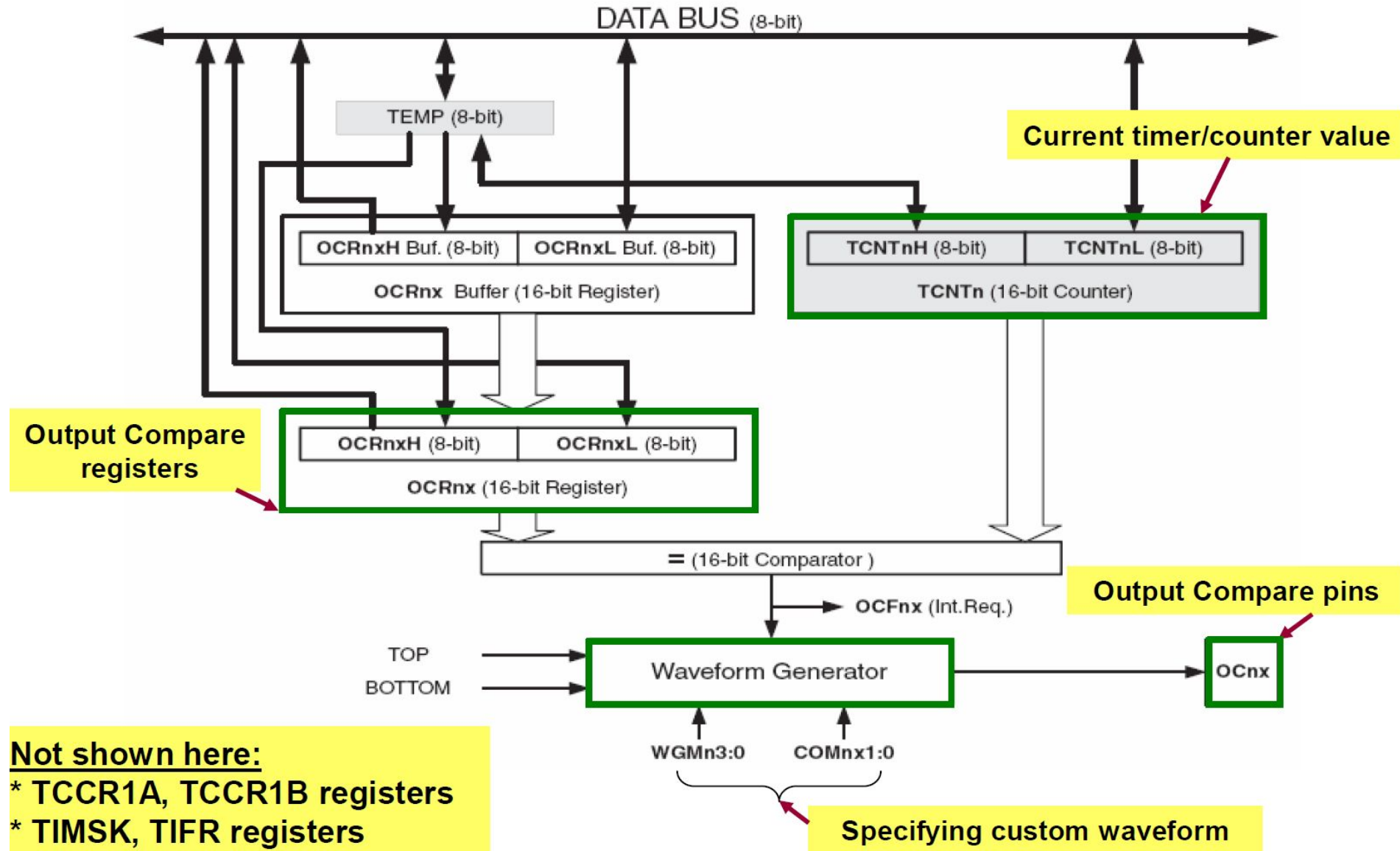
OUTPUT COMPARE UNIT IN TIMER 1

- Timer 1 has two output compare channels: A and B
- Timer 1 is continuously compared to OCR1A or OCR1B or a fixed limit
- When a match occurs, a flag OCF1x is set
 - where x = 'A' or 'B'
- When a match occurs, Timer 1 can
 - trigger an output compare interrupt
 - change output compare pins OC1x
- PD4 and PD5 must be enabled for output



OUTPUT COMPARE UNIT – BLOCK

DIAGRAM



RELEVANT PINS

(XCK/T0) PB0	☐	1	40	☐	PA0 (ADC0)
(T1) PB1	☐	2	39	☐	PA1 (ADC1)
(INT2/AIN0) PB2	☐	3	38	☐	PA2 (ADC2)
(OC0/AIN1) PB3	☐	4	37	☐	PA3 (ADC3)
($\overline{SS}$) PB4	☐	5	36	☐	PA4 (ADC4)
(MOSI) PB5	☐	6	35	☐	PA5 (ADC5)
(MISO) PB6	☐	7	34	☐	PA6 (ADC6)
(SCK) PB7	☐	8	33	☐	PA7 (ADC7)
RESET	☐	9	32	☐	AREF
VCC	☐	10	31	☐	GND
GND	☐	11	30	☐	AVCC
XTAL2	☐	12	29	☐	PC7 (TOSC2)
XTAL1	☐	13	28	☐	PC6 (TOSC1)
(RXD) PD0	☐	14	27	☐	PC5 (TDI)
(TXD) PD1	☐	15	26	☐	PC4 (TDO)
(INT0) PD2	☐	16	25	☐	PC3 (TMS)
(INT1) PD3	☐	17	24	☐	PC2 (TCK)
(OC1B) PD4	☐	18	23	☐	PC1 (SDA)
(OC1A) PD5	☐	19	22	☐	PC0 (SCL)
(ICP1) PD6	☐	20	21	☐	PD7 (OC2)

PORTD pins must be enabled for output



RELATED REGISTERS – TIMER1

1. TIMER/COUNTER1

- TCNT1
- 16 bit register that stores the **current value** of the counter

2. Timer/Counter 1 Control Registers

- TCCR1A and TCCR1B
- To **configure** the operations of Timer 1.

3. Interrupt registers

- TIMSK to **enable** timer interrupts
- TIFR to monitor **status** of timer interrupts

RELATED REGISTERS – TIMER1

4. Input Capture Register

- ICR1

- to store timer value when an event occurs on input capture pin

5. Output Compare Registers

- OCR1A, OCR1B

- To store the preset values for output compare

TIMER/COUNTER 1 CONTROL

REGISTER A (TCR1A)
7 6 5 4 3 2 1 0

COM1A1	COM1A0	COM1B1	COM1B0	FOC1A	FOC1B	WGM11	WGM10
--------	--------	--------	--------	-------	-------	-------	-------

Specify Waveform Generation Mode
used with WGM13, WGM12
WGM13..WGM10 = 0000 for normal mode.

1 to force output compare on channel B

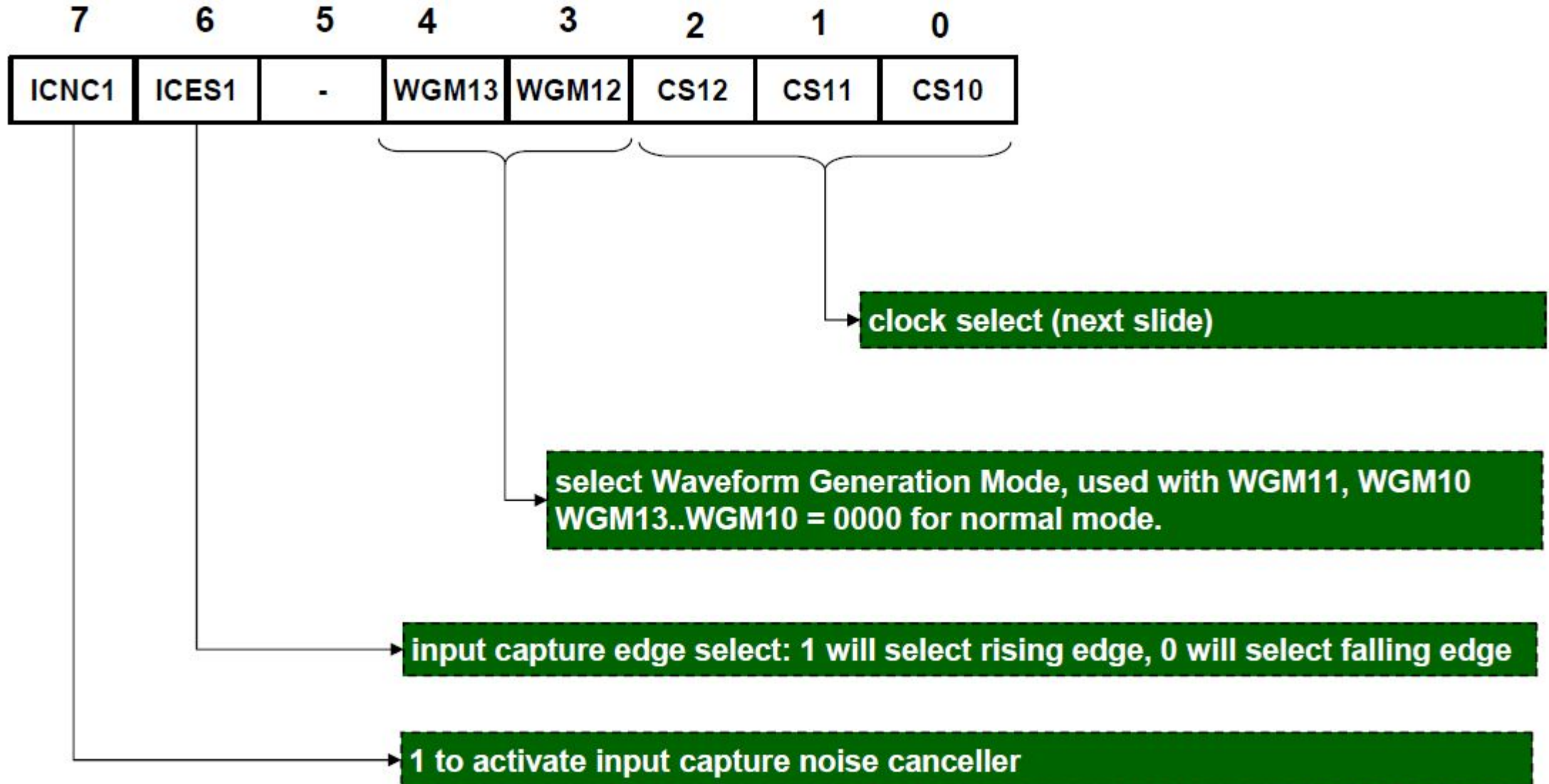
1 to force output compare on channel A

Output compare mode for channel B

Output compare mode for channel A



TIMER/COUNTER 1 CONTROL REGISTER B (TCCR1B)



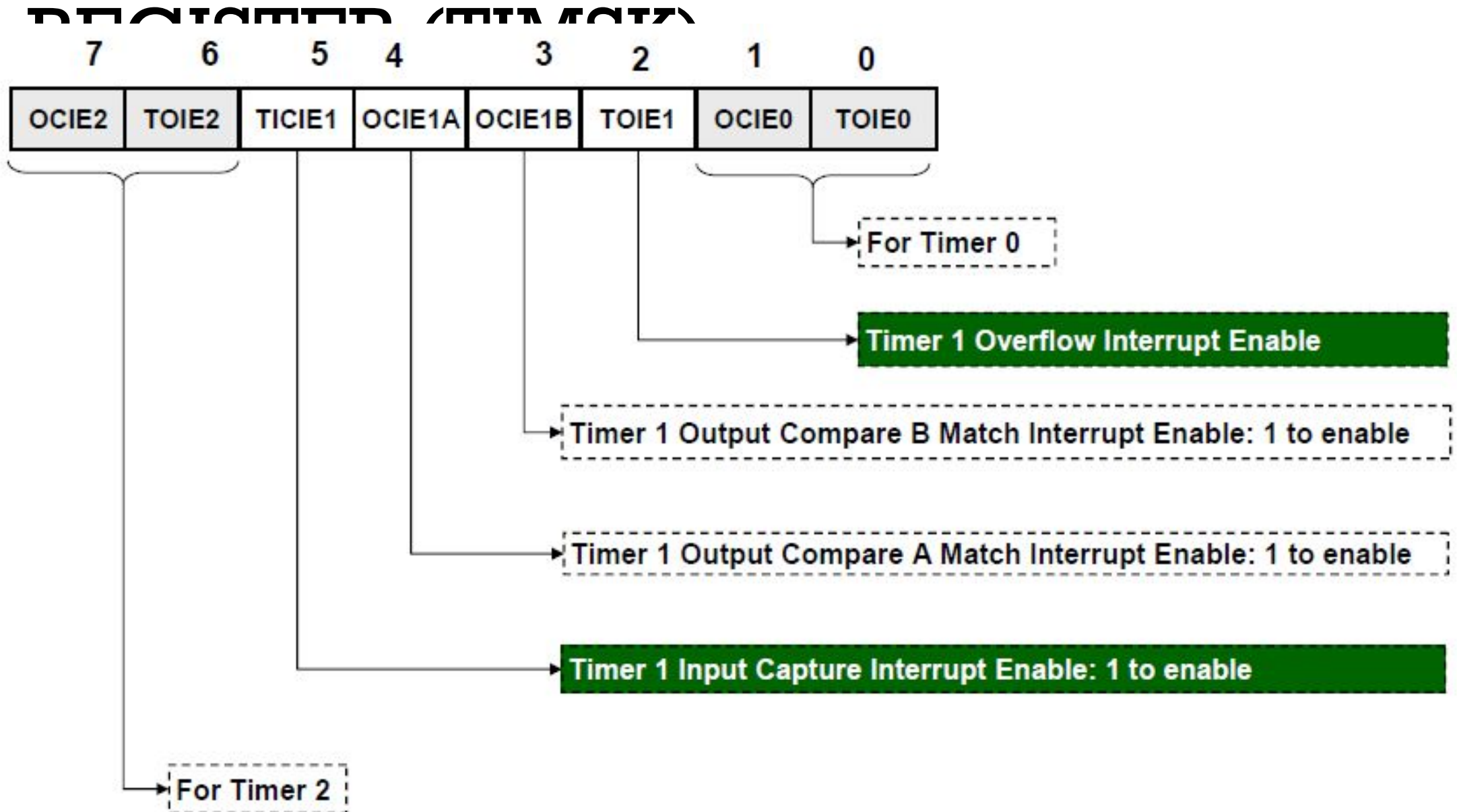
CLOCK SELECT

CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	$\text{clk}_{\text{I/O}}/1$ (No prescaling)
0	1	0	$\text{clk}_{\text{I/O}}/8$ (From prescaler)
0	1	1	$\text{clk}_{\text{I/O}}/64$ (From prescaler)
1	0	0	$\text{clk}_{\text{I/O}}/256$ (From prescaler)
1	0	1	$\text{clk}_{\text{I/O}}/1024$ (From prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

- For ATmega16, the internal clock is set by default at $\text{clk}_{\text{I/O}} = 1\text{MHz}$.
- Timer 1 can run using the internal or external clock.
- If using the internal clock, we can set Timer 1 to run at a speed that is 8, 64, 256 or 1024 times slower than the internal clock.

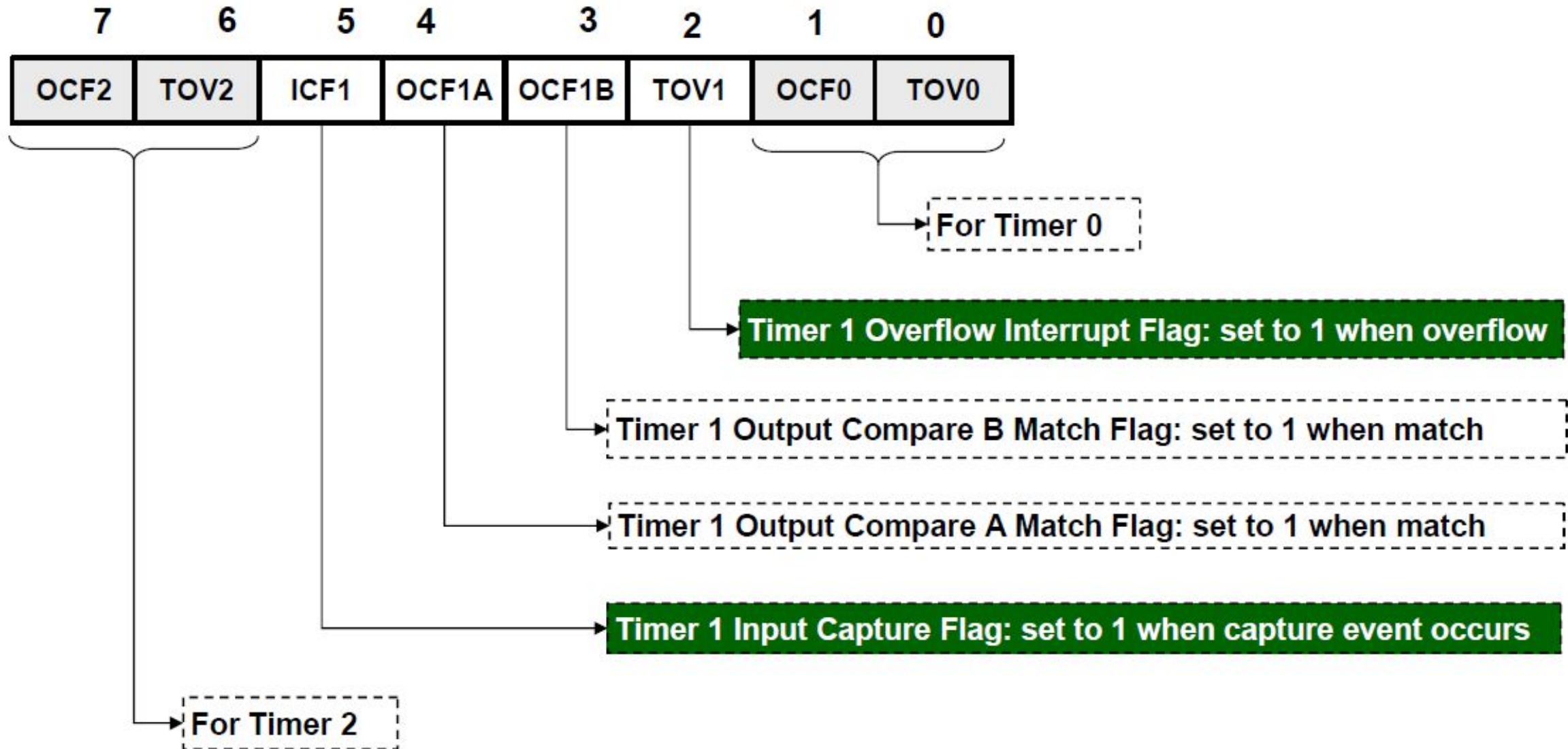


TIMER/COUNTER INTERRUPT MASK



TIMER/COUNTER INTERRUPT FLAG

DECODED (TTED)



TIMER RELATED INTERRUPTS

4	TIMER2_COMP_vect	Timer/Counter2 Compare Match
5	TIMER2_OVF_vect	Timer/Counter2 Overflow
6	TIMER1_CAPT_vect	Timer/Counter1 Capture Event
7	TIMER1_COMPA_vect	Timer/Counter1 Compare Match A
8	TIMER1_COMPB_vect	Timer/Counter1 Compare Match B
9	TIMER1_OVF_vect	Timer/Counter1 Overflow
10	TIMER0_OVF_vect	Timer/Counter0 Overflow



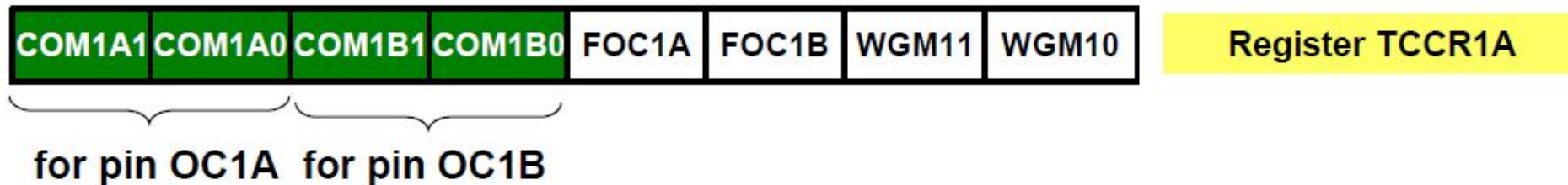
MAIN ASPECTS

- What **changes** can be made to output compare pins OC1x?
- What are the available **operation modes** of timer 1?
- Steps to produce a custom waveform?



CHANGING OUTPUT COMPARE PINS OC1X

- When there's a timer event (compare match or timer reaching 0), pins OC1x can be automatically updated: toggle, set to 1, clear to 0, or no change.
- The type of update is specified by two bits in TCCR1A register: COM1x1 and COM1x2 where x = 'A' or 'B'.



- The exact change will also depend on the operation mode of Timer 1.



OPERATIONS MODES OF TIMER 1

- Timer 1 supports 15 operation modes, which can be divided into 5 groups:

- Normal

- Clear Timer on Compare Match (CTC Mode)

- Fast PWM

- Phase correct PWM

- Phase and Frequency Correct PWM

} Three PWM groups

- The operation mode is selected by 4 bits:

$WGM = \{WGM13, WGM12, WGM11, WGM10\}$

- Each groups of operations will be discussed next.



SOME DEFINITIONS

BOTTOM	The counter reaches the <i>BOTTOM</i> when it becomes 0x0000.
MAX	The counter reaches its <i>MAX</i> imum when it becomes 0xFFFF (decimal 65535).
TOP	The counter reaches the <i>TOP</i> when it becomes equal to the highest value in the count sequence. The TOP value can be assigned to be one of the fixed values: 0x00FF, 0x01FF, or 0x03FF, or to the value stored in the OCR1A or ICR1 Register. The assignment is dependent of the mode of operation.



SELECTING OPERATION MODE OF TIMER 1

COM1A1	COM1A0	COM1B1	COM1B0	FOC1A	FOC1B	WGM11	WGM10
--------	--------	--------	--------	-------	-------	-------	-------

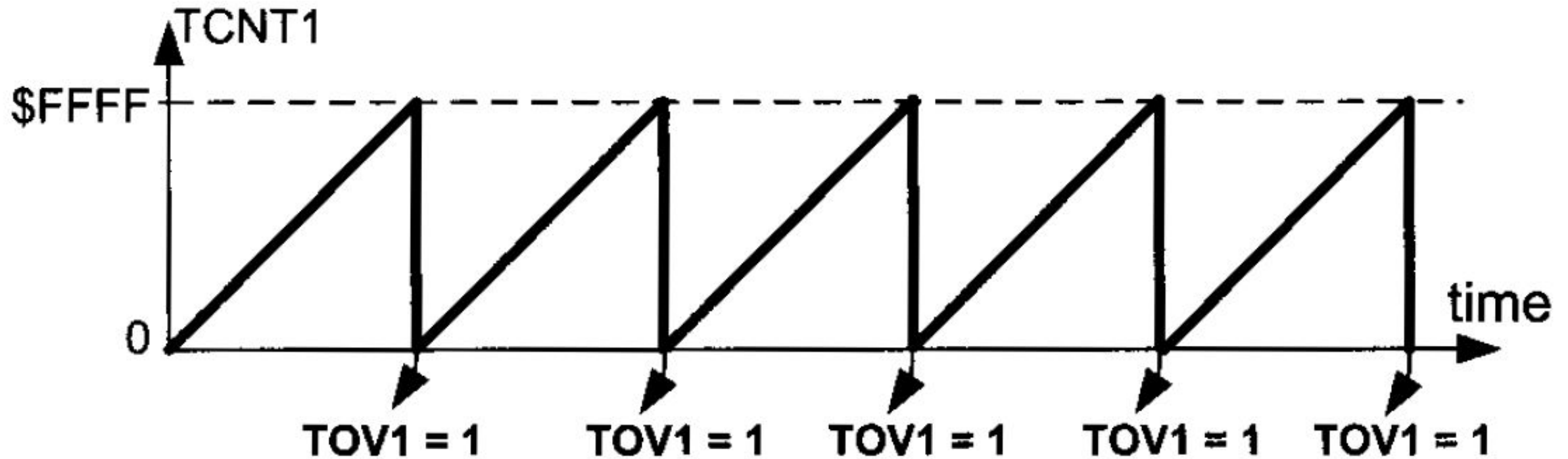
Register TCCR1A

ICNC1	ICES1	-	WGM13	WGM12	CS12	CS11	CS10
-------	-------	---	-------	-------	------	------	------

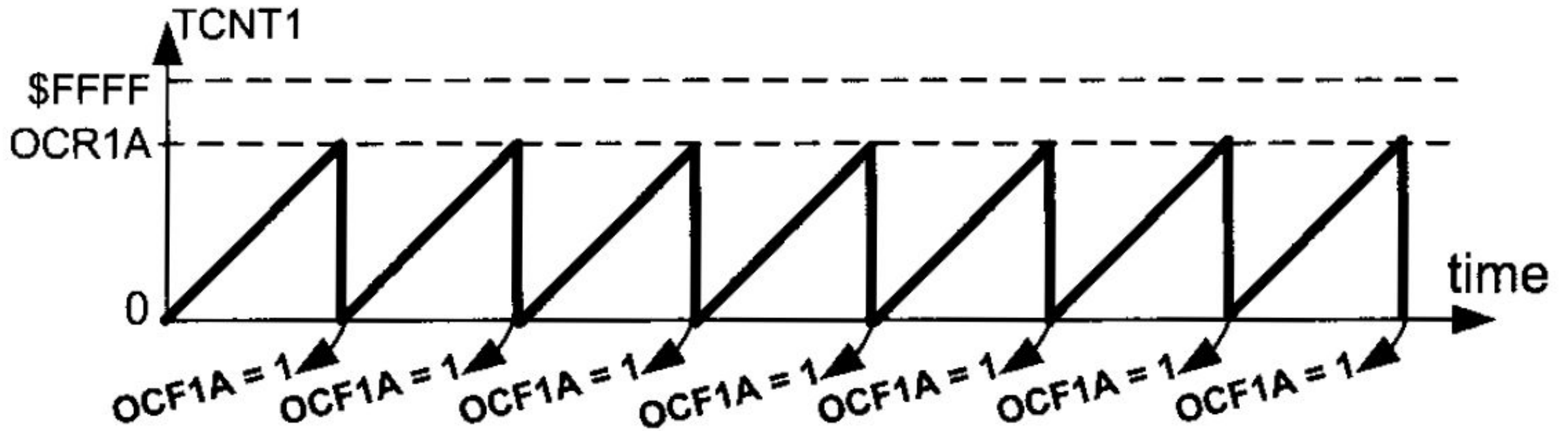
Register TCCR1B

Mode	WGM13	WGM12 (CTC1)	WGM11 (PWM11)	WGM10 (PWM10)	Timer/Counter Mode of Operation	TOP	Update of OCR1X	TOV1 Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, Phase Correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, Phase Correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, Phase Correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCR1A	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	BOTTOM	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	BOTTOM	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	BOTTOM	TOP
8	1	0	0	0	PWM, Phase and Frequency Correct	ICR1	BOTTOM	BOTTOM
9	1	0	0	1	PWM, Phase and Frequency Correct	OCR1A	BOTTOM	BOTTOM
10	1	0	1	0	PWM, Phase Correct	ICR1	TOP	BOTTOM
11	1	0	1	1	PWM, Phase Correct	OCR1A	TOP	BOTTOM
12	1	1	0	0	CTC	ICR1	Immediate	MAX
13	1	1	0	1	Reserved	–	–	–
14	1	1	1	0	Fast PWM	ICR1	BOTTOM	TOP
15	1	1	1	1	Fast PWM	OCR1A	BOTTOM	TOP

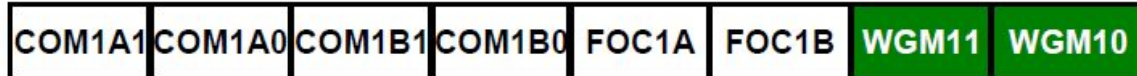
NORMAL MODE (WGM13:10 = 0000)



CTC MODE (WGM13:10 = 0100)



CTC MODE



Register TCCR1A

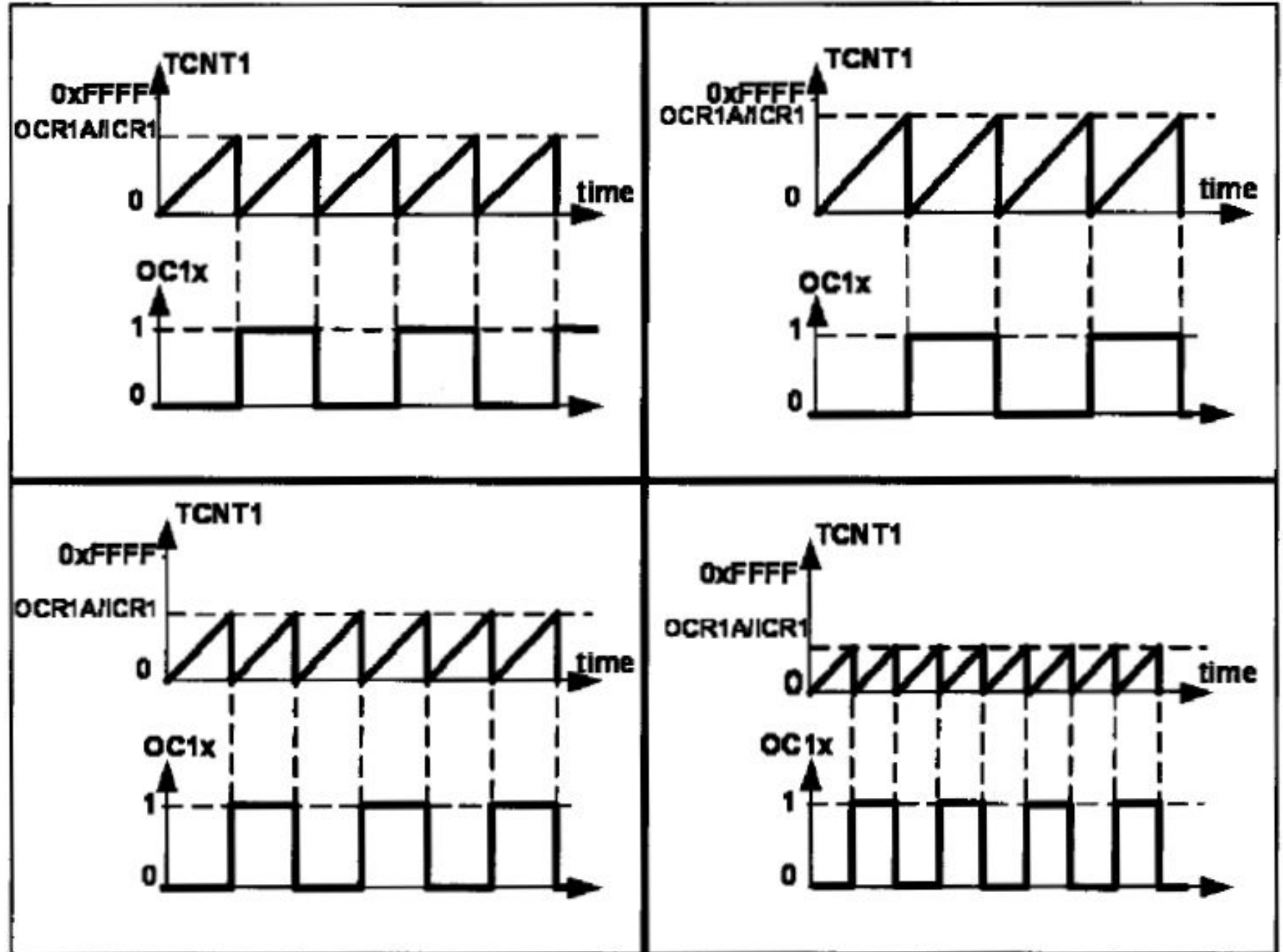
- On compare match, change of pins OC1x is allowed.

COM1A1/ COM1B1	COM1A0/ COM1B0	Description
0	0	Normal port operation. OC1A/OC1B disconnected.
0	1	Toggle OC1A/OC1B on compare match.
1	0	Clear OC1A/OC1B on compare match. (Set output to low level).
1	1	Set OC1A/OC1B on compare match. (Set output to high level).

Changing OC1x in CTC mode



CTC MODES



CTC MODES

WGM13:10 = 0100

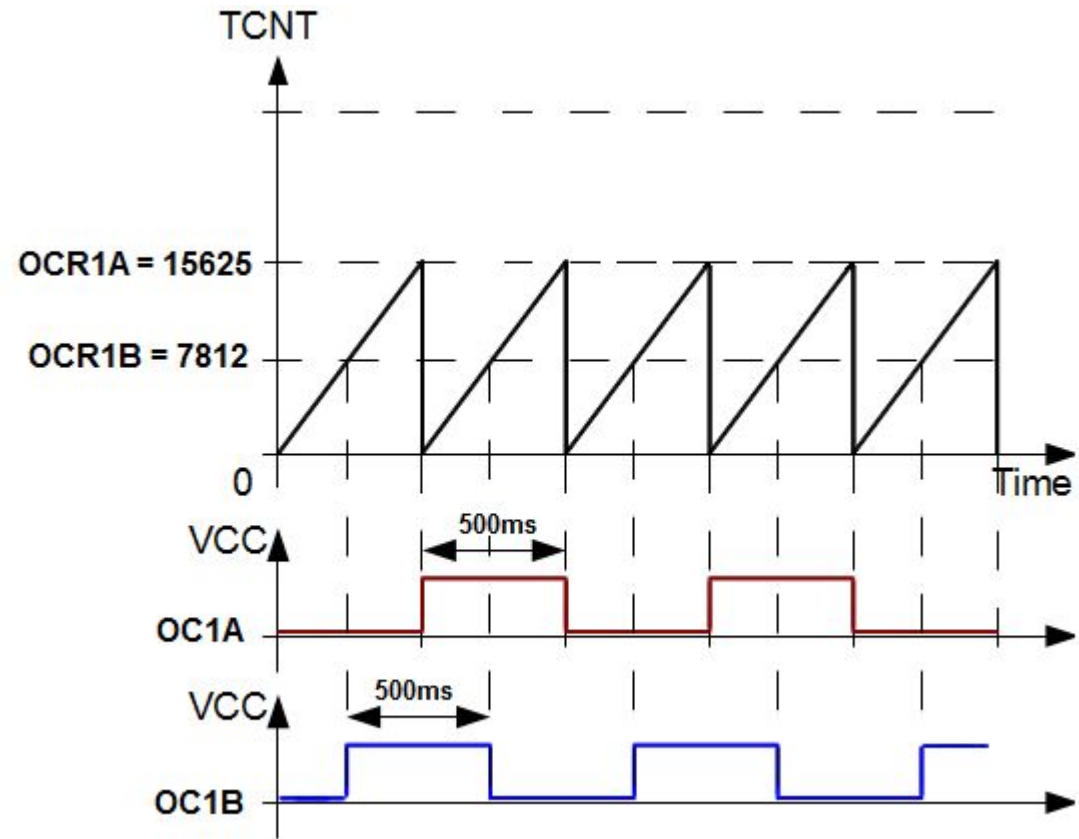
OCR1A = 15625

OCR1B = 7812

COM1A1:COM1A0 = 01

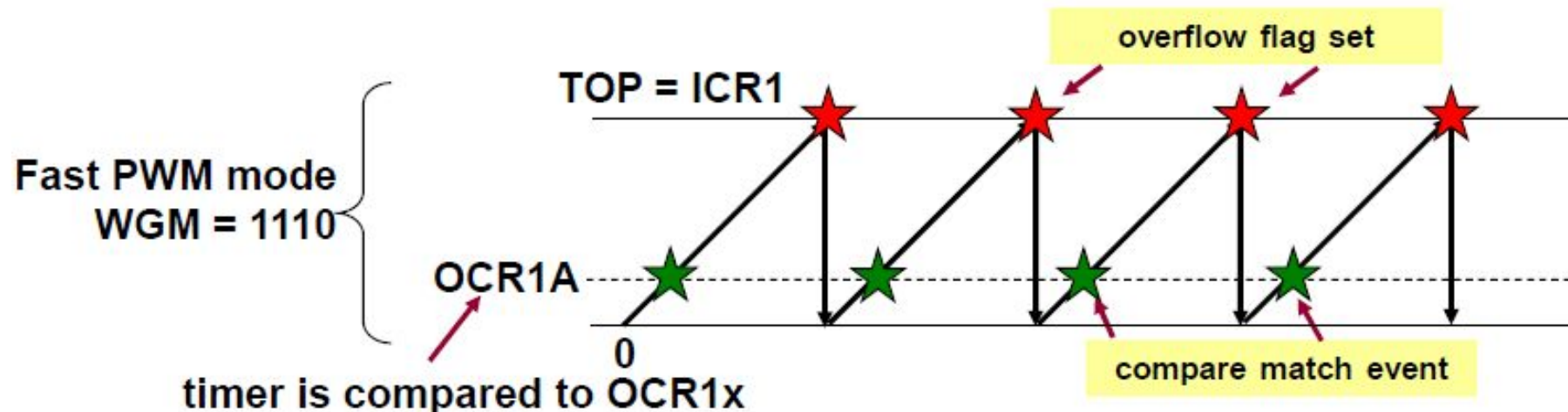
COM1B1:COM1B0 = 01

Counter is cleared on match with OCR1A.
Counter is continue on match with OCR1B



FAST PWM MODES

- Timer goes from 0 to TOP, where TOP is equal to
 - 0xFF (for 8-bit mode, WGM = 0101),
 - 0x1FF (for 9-bit mode, WGM = 0110),
 - 0x3FF (for 10-bit mode, WGM = 0111),
 - value in ICR1 (for WGM = 1110),
 - value in OCR1A (for WGM = 1111).
- Compare match occurs when timer = OCR1x register.



Mode	WGM13	WGM12 (CTC1)	WGM11 (PWM11)	WGM10 (PWM10)	Timer/Counter Mode of Operation	TOP	Update of OCR1X	TOV1 Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, Phase Correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, Phase Correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, Phase Correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCR1A	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	BOTTOM	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	BOTTOM	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	BOTTOM	TOP
8	1	0	0	0	PWM, Phase and Frequency Correct	ICR1	BOTTOM	BOTTOM
9	1	0	0	1	PWM, Phase and Frequency Correct	OCR1A	BOTTOM	BOTTOM
10	1	0	1	0	PWM, Phase Correct	ICR1	TOP	BOTTOM
11	1	0	1	1	PWM, Phase Correct	OCR1A	TOP	BOTTOM
12	1	1	0	0	CTC	ICR1	Immediate	MAX
13	1	1	0	1	Reserved	–	–	–
14	1	1	1	0	Fast PWM	ICR1	BOTTOM	TOP
15	1	1	1	1	Fast PWM	OCR1A	BOTTOM	TOP



FAST PWM MODES

- On compare match, change of pins OC1x is allowed.

COM1A1/ COM1B1	COM1A0/ COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	- WGM13:0=15: Toggle OC1A on Compare Match, OC1B disconnected. - For other WGM13:0 settings, normal port operation, OCnA/OCnB disconnected.
1	0	Clear OC1A/OC1B on compare match. Set OC1A/OC1B at BOTTOM.
1	1	Set OC1A/OC1B on compare match. Clear OC1A/OC1B at BOTTOM. (inverting mode)

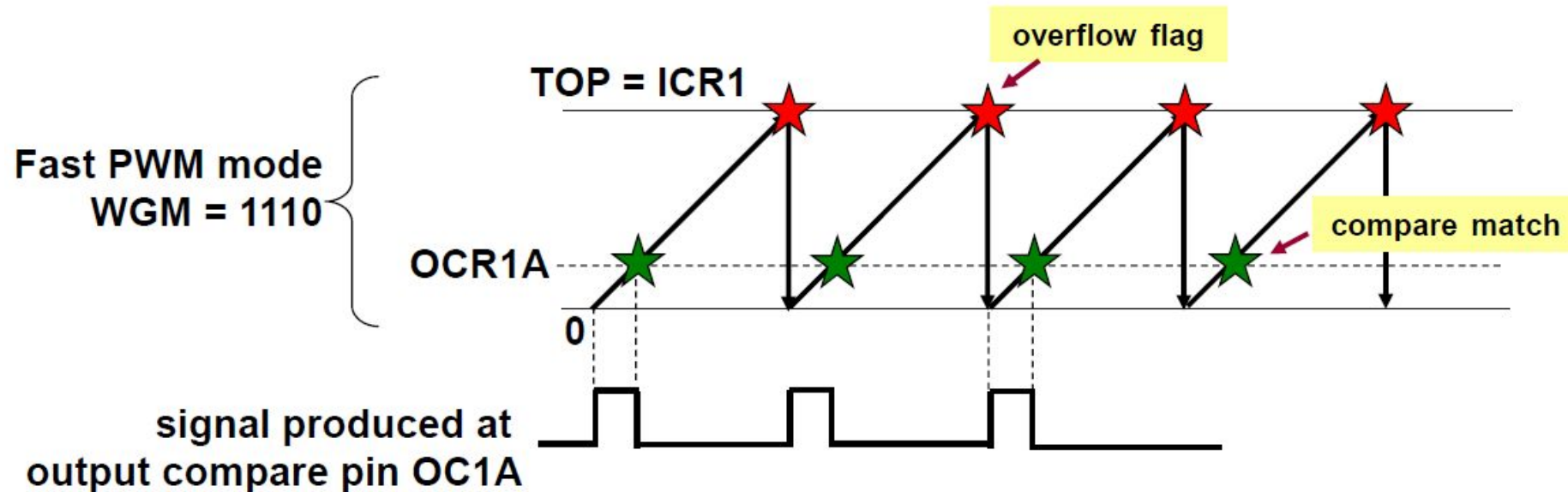
Changing OC1x in fast PWM mode
(Note that BOTTOM = 0)



FAST PWM MODES

Used in
the
example
below →

COM1A1/ COM1B1	COM1A0/ COM1B0	Description
0	0	-
0	1	-
1	0	Clear OC1A/OC1B on compare match Set OC1A/OC1B at BOTTOM
1	1	-

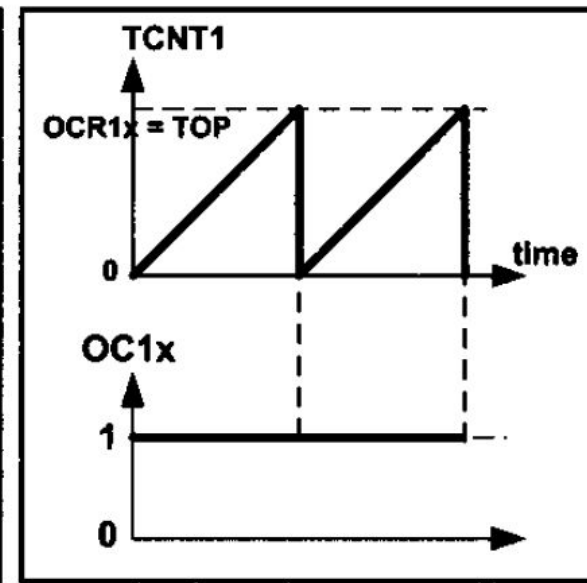
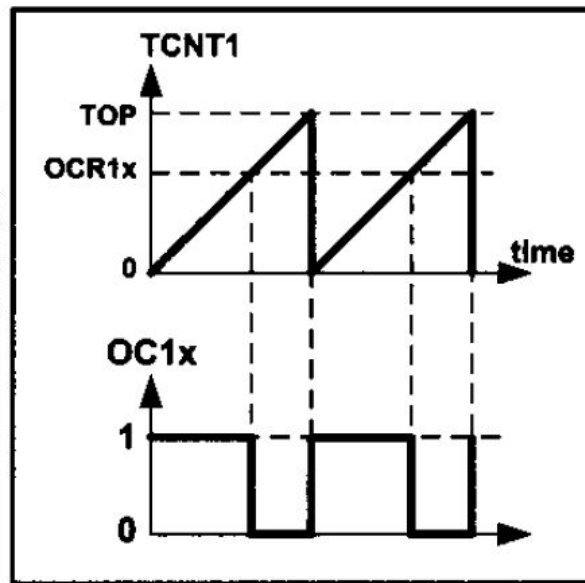
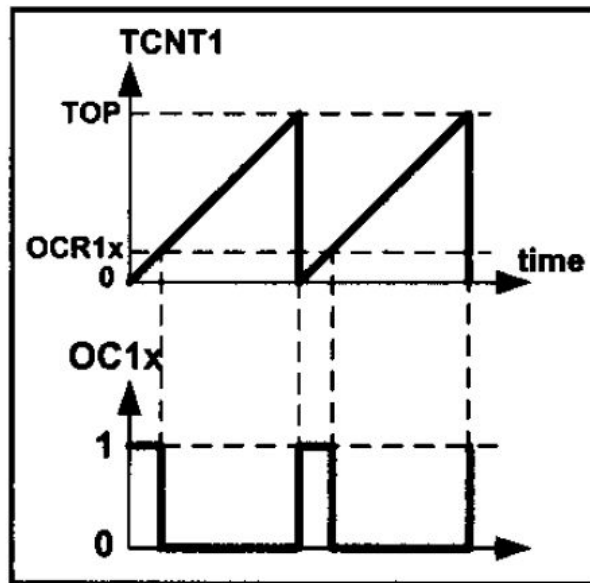


FAST PWM MODES

1

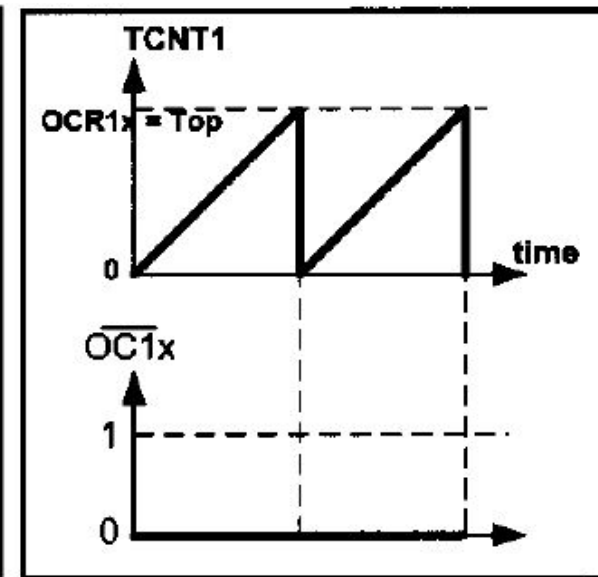
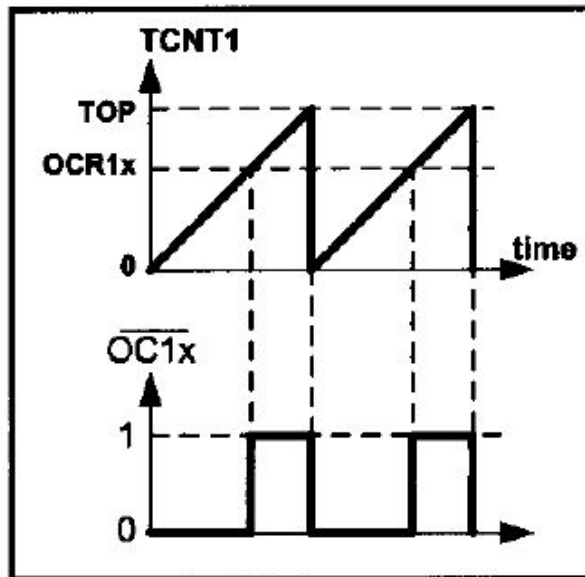
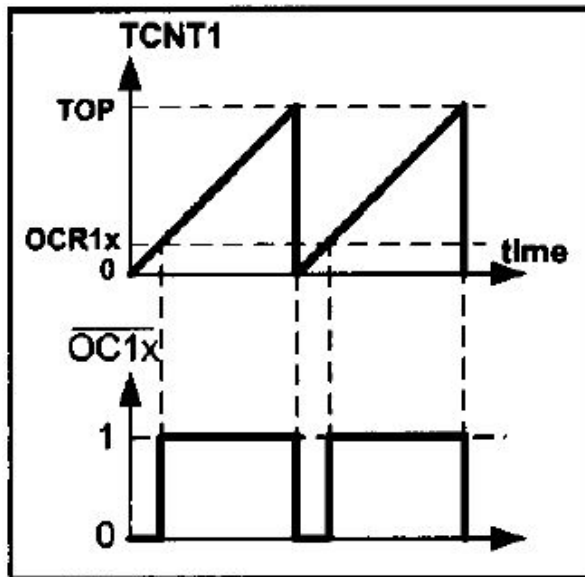
0

Clear OC1A/OC1B on compare match.
Set OC1A/OC1B at **BOTTOM**.



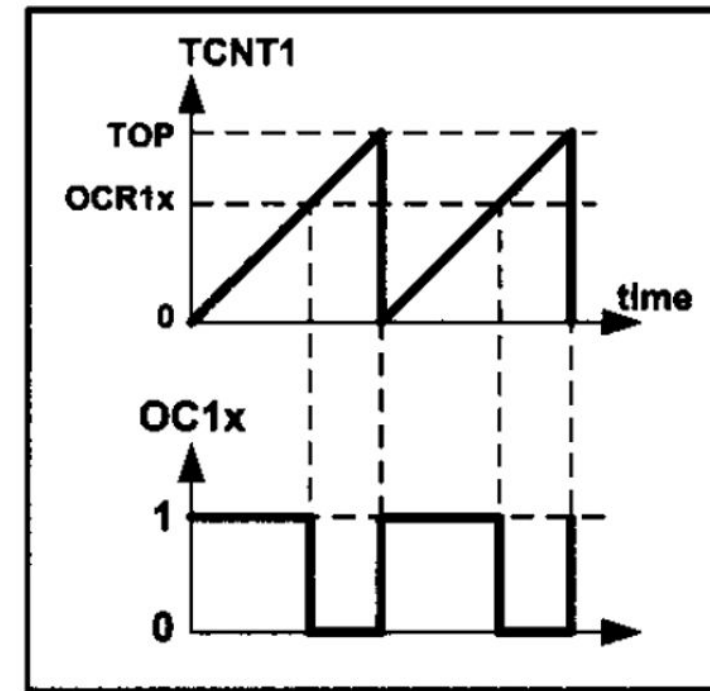
FAST PWM MODES

1	1	Set OC1A/OC1B on compare match. Clear OC1A/OC1B at BOTTOM . (inverting mode)
---	---	---



FAST PWM MODES

- What is the frequency?



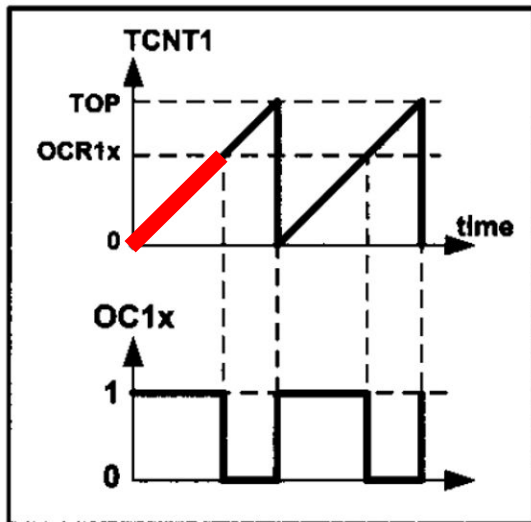
$$\left. \begin{aligned} F_{\text{generated wave}} &= \frac{F_{\text{timer clock}}}{\text{Top} + 1} \\ F_{\text{timer clock}} &= \frac{F_{\text{oscillator}}}{N} \end{aligned} \right\} \Rightarrow F_{\text{generated wave}} = \frac{F_{\text{oscillator}}}{(\text{Top} + 1) \times N}$$



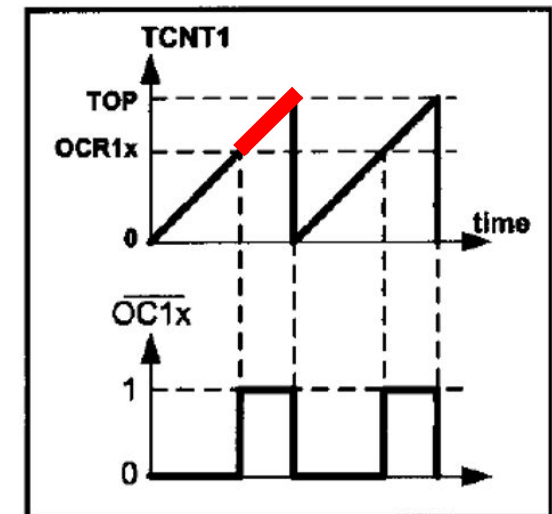
FAST PWM MODES

- What is the duty cycle?

$$\text{Duty Cycle} = \frac{\text{OCR1x} + 1}{\text{Top} + 1} \times 100$$

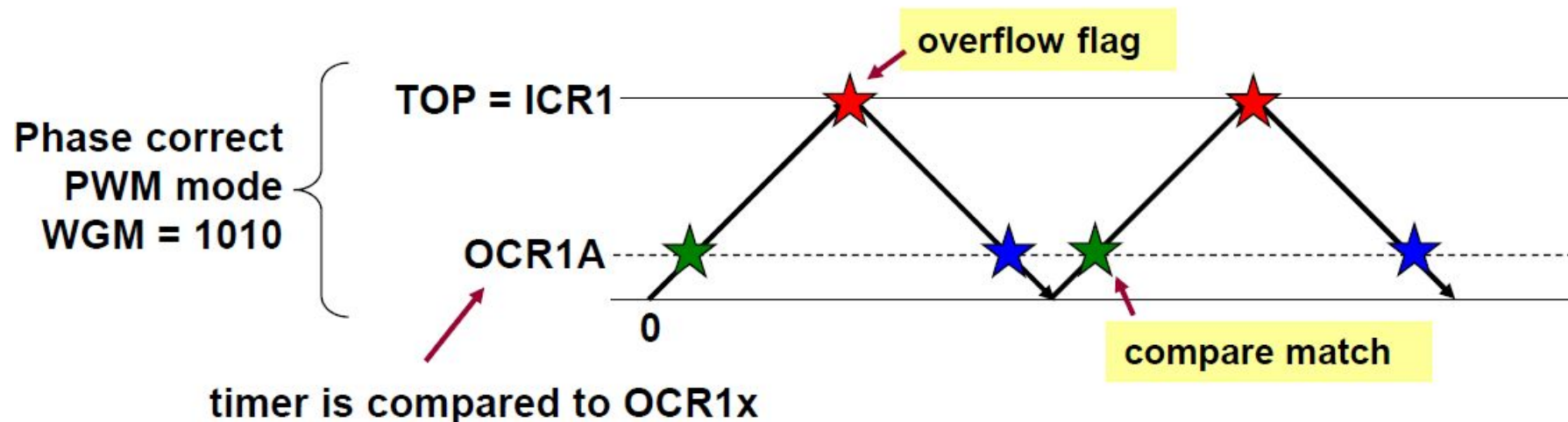


$$\text{Duty Cycle} = \frac{\text{Top} - \text{OCR1x}}{\text{Top} + 1} \times 100$$



PHASE CORRECT PWM MODES

- Timer counts up and down between 0 and TOP, where TOP is equal to
 - 0xFF (for 8-bit mode, WGM = 1000)
 - 0x1FF (for 9-bit mode, WGM = 0010)
 - 0x3FF (for 10-bit mode, WGM = 0011)
 - value in ICR1 (for WGM = 1010)
 - value in OCR1A (for WGM = 1011)
- Compare match occurs when timer = OCR1x register.



PHASE CORRECT PWM

MODE

- On compare match, change of pins OC1x is allowed.

COM1A1/ COM1B1	COM1A0/ COM1B0	Description
0	0	Normal port operation, OC1A/OC1B disconnected.
0	1	- WGM13:0= 9 or 14: Toggle OCnA on Compare Match, OCnB disconnected. - For other WGM13:0 settings, normal port operation, OC1A/OC1B disconnected.
1	0	Clear OC1A/OC1B on compare match when up-counting. Set OC1A/OC1B on compare match when down-counting.
1	1	Set OC1A/OC1B on compare match when up-counting. Clear OC1A/OC1B on compare match when down-counting.

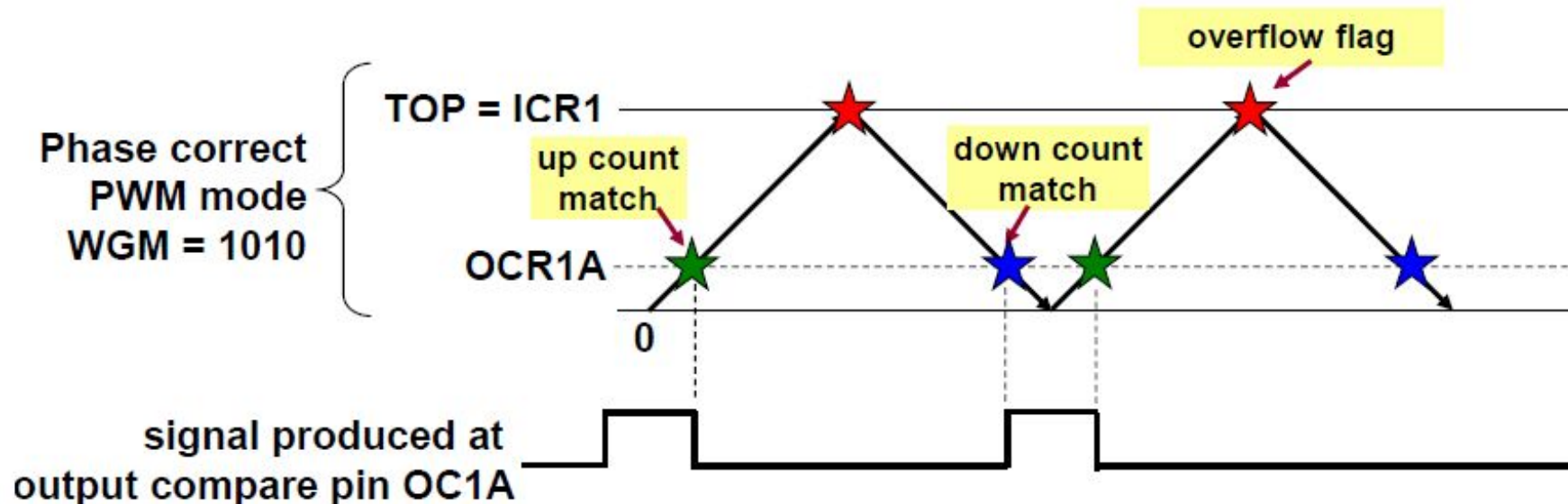
Changing OC1x in Phase Correct PWM mode



PHASE CORRECT PWM MODES

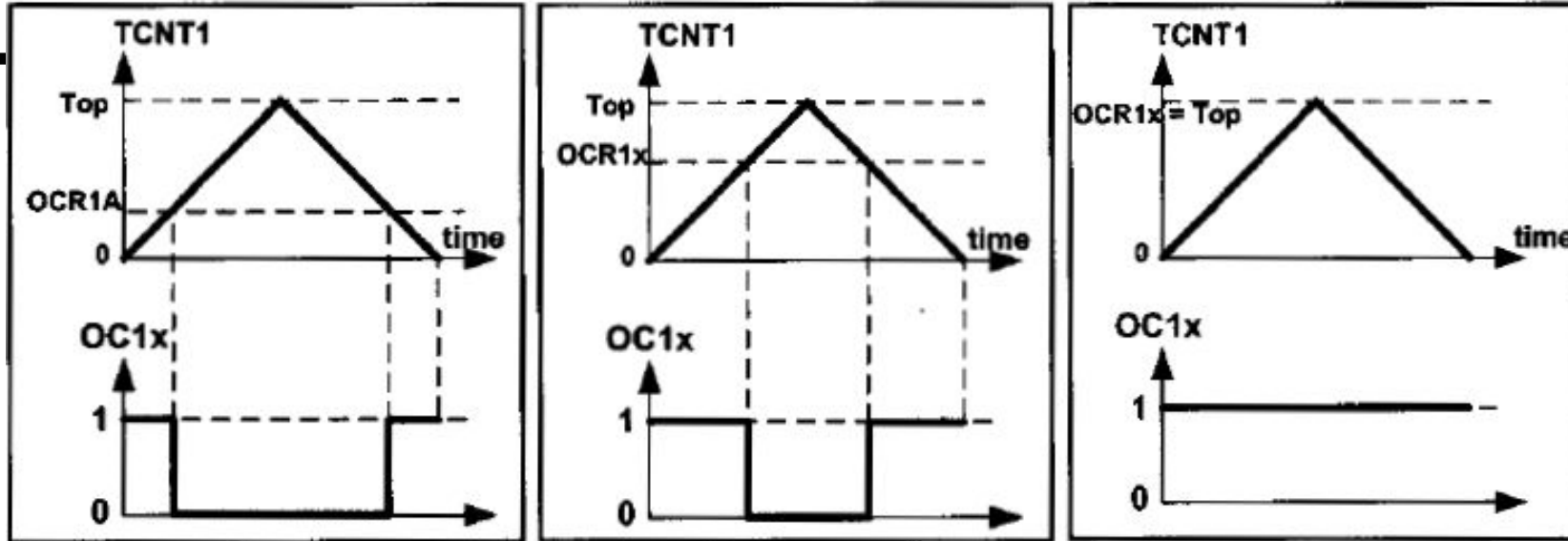
Used in
the
example
below →

COM1A1/ COM1B1	COM1A0/ COM1B0	Description
0	0	-
0	1	-
1	0	Clear OC1A/OC1B on compare match when up-counting. Set OC1A/OC1B on compare match when down-counting.
1	1	-

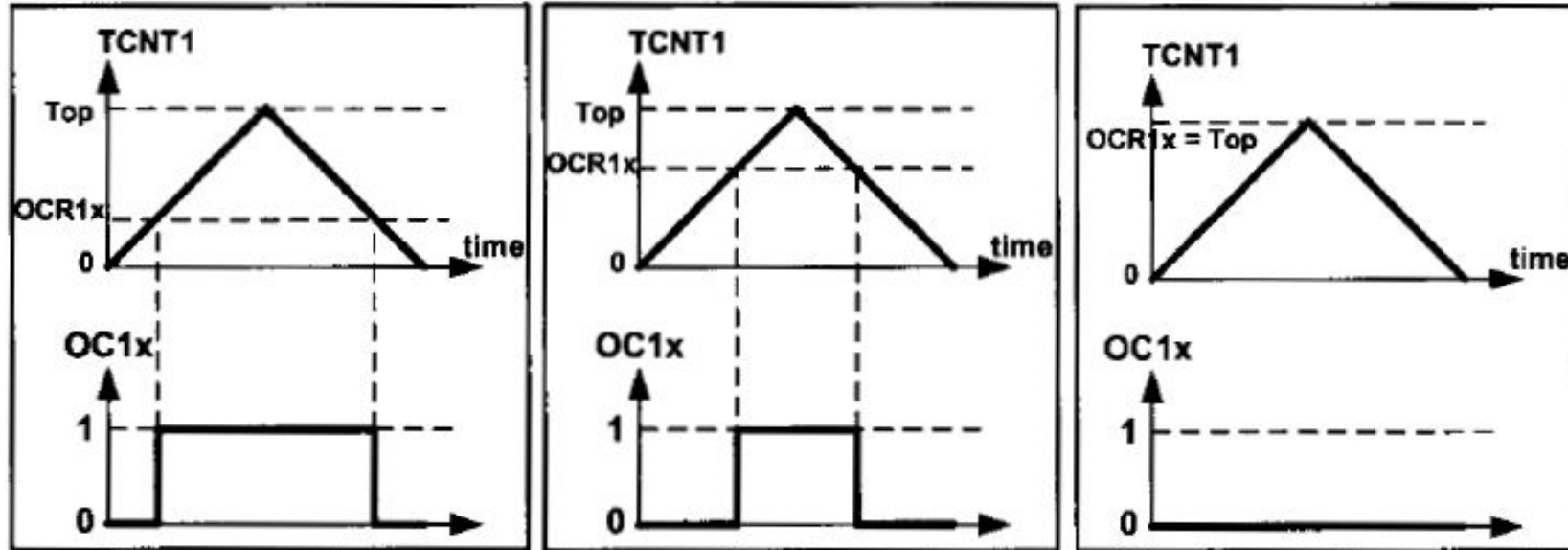


PHASE CORRECT PWM

MODE 1



PHASE CORRECT PWM MODES



PHASE CORRECT PWM MODES

- Self Study
 - Frequency
 - Duty cycle
 - AVR Microcontroller & Embedded Systems: Chapter 16

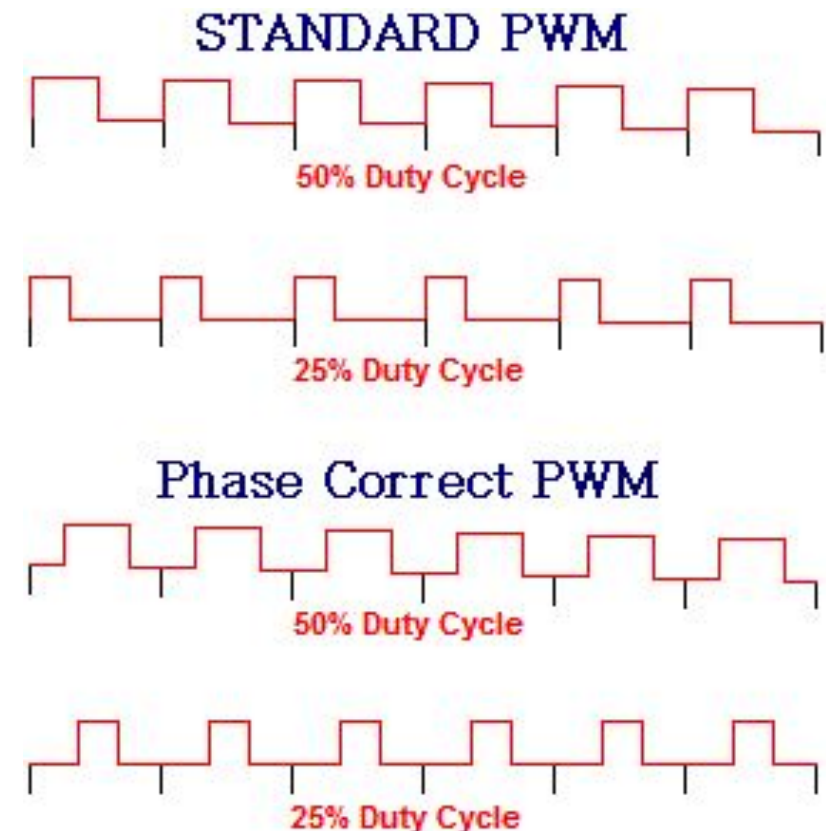






PHASE CORRECT VS FAST PWM

- The maximum possible frequency of the Fast PWM output is 2X that of the phase correct PWM
- In Fast PWM, the phase of the wave is different for different duty cycles
 - while it remains unchanged in the Phase correct PWM



PHASE CORRECT VS FAST PWM

- Phase Correct PWM
 - Preferred for motor control
 - Due to the symmetric feature
- Fast PWM
 - Where the phase change does not matter
 - Need to generate waves with high frequencies.



PRODUCING A CUSTOM WAVEFORM

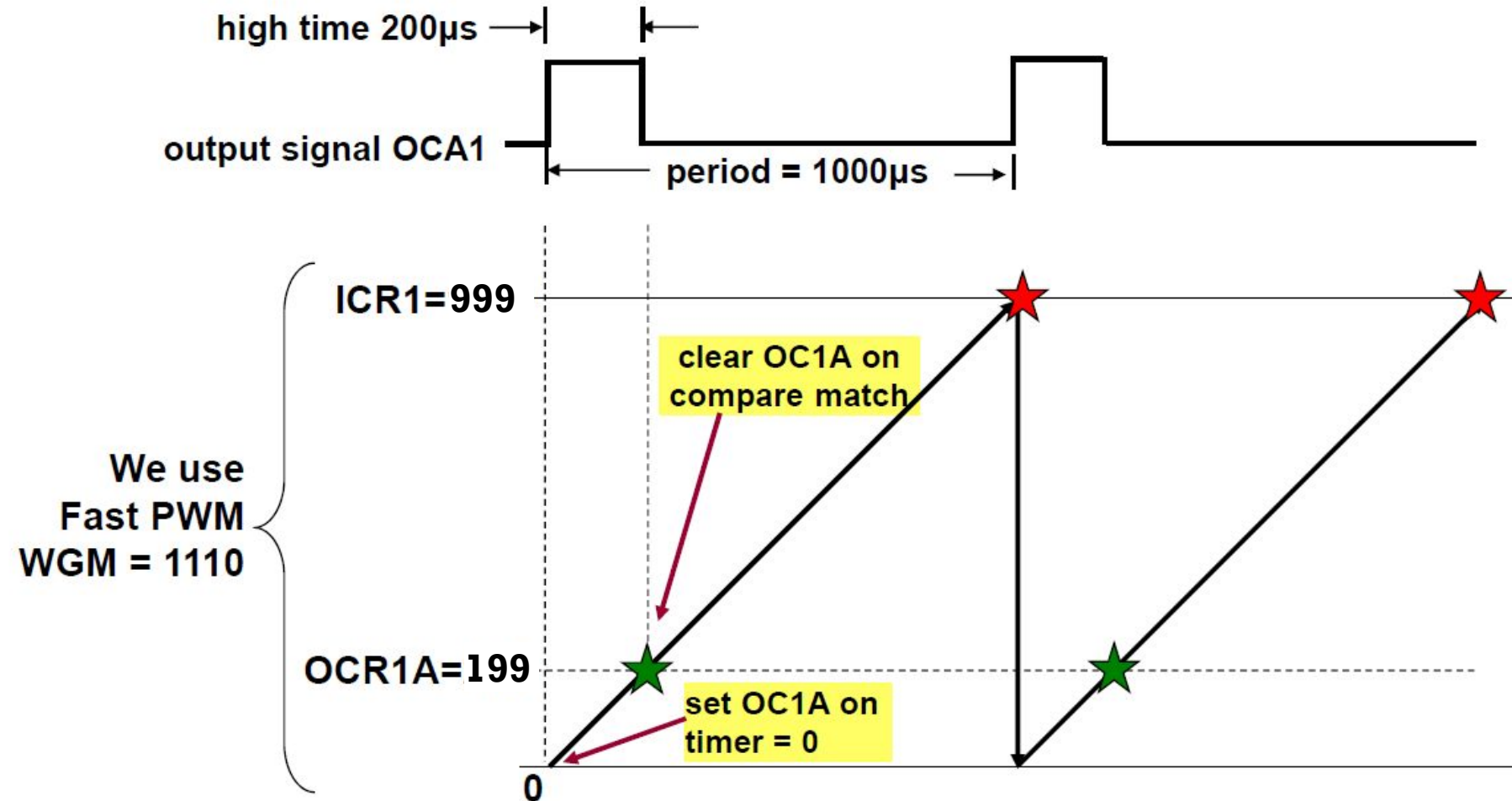
Steps to produce a custom waveform on an output compare pin OC1x

- Select the operation mode of Timer 1: CTC, fast PWM, or phase correct PWM, ...
 - Select how output compare pin will be updated on compare match event.
 - Configure timer 1: clock source, prescaler, ...
 - Put correct values in the output compare registers.
- set registers
TCCR1A and
TCCR1B
- set register
OCR1A or ICR1



PRODUCING A CUSTOM WAVEFORM

Use Timer 1 to create a signal with period = $1000\mu\text{s}$, high time = $200\mu\text{s}$.



PRODUCING A CUSTOM WAVEFORM

COM1A1	COM1A0	COM1B1	COM1B0	FOC1A	FOC1B	WGM11	WGM10	Register TCCR1A
1	0	0	0	0	0	1	0	

ICNC1	ICES1	-	WGM13	WGM12	CS12	CS11	CS10	Register TCCR1B
0	0	0	1	1	0	0	1	

- $ICR1 = 999$ → period of output signal
- $OCR1A = 199$ → pulse width of output signal
- $WGM3:0 = 1110$ → Fast PWM mode where $TOP = ICR1$.
- $CS12:0 = 001$ → Internal clock, no prescaler
- $COM1A1:0 = 10$ → set OC1A when timer = 0
clear OC1A when compare match



```
#include <avr\io.h>

int main(void) {

    DDRD=0b00100000; // set port D for output (D.5 is OC1A)

    // Set register TCCR1A
    // WGM11:WGM10      = 10: with WGM13-WGM12 to select timer mode 1110
    //                               Fast PWM, timer 1 runs from 0 to ICR1
    // COM1A1:COM1A0    = 10: clear OC1A when compare match, set OC1A when 0
    //                               compare match occurs when timer = OCR1A
    TCCR1A = 0b10000010;

    // Set register TCCR1B
    // WGM13:WGM12      = 11
    // CS12:CS0         = 001: internal clock 1MHz, no prescaler
    TCCR1B = 0b00011001;

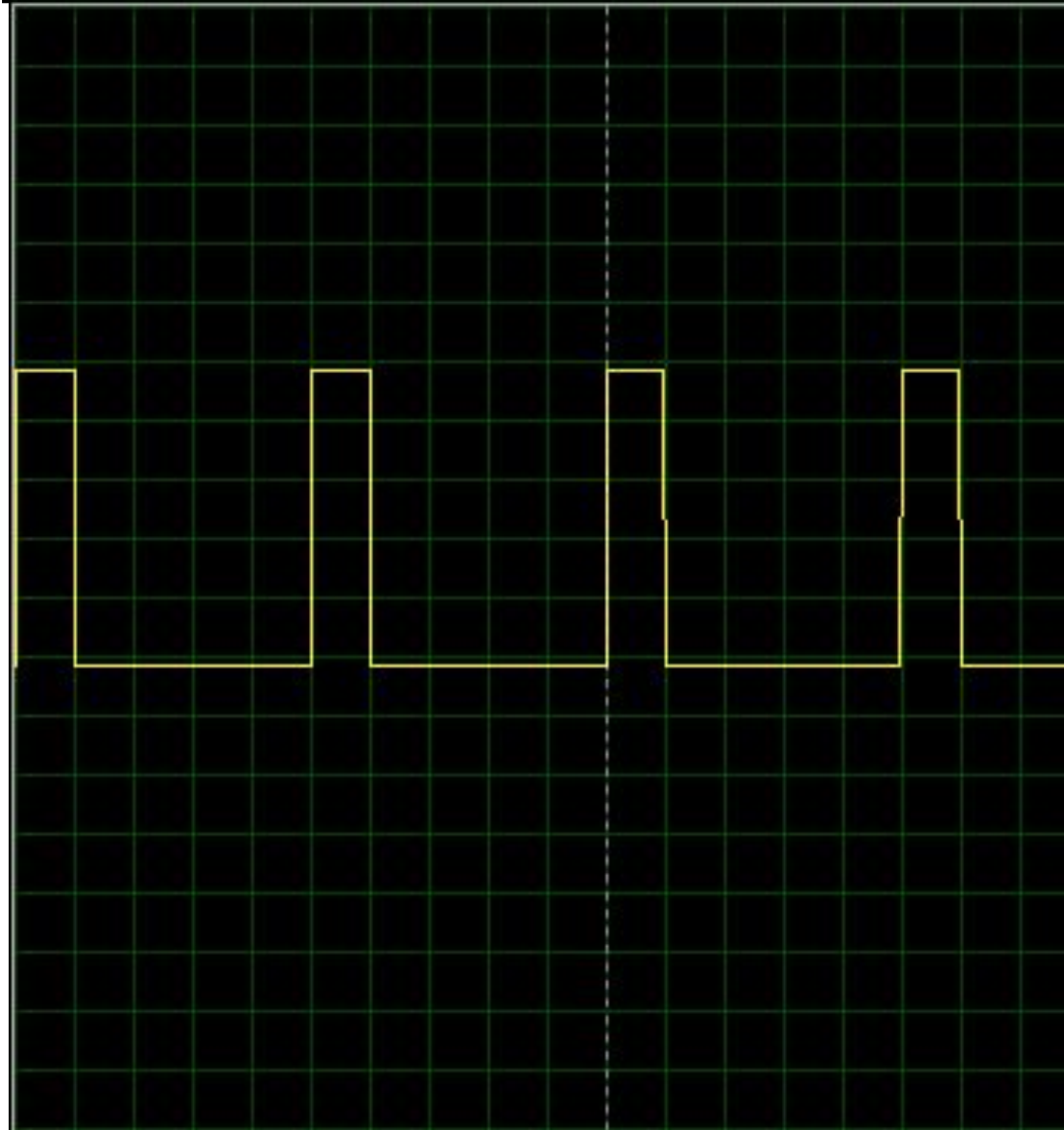
    ICR1  = 999 ;      // period of output signal
    OCR1A = 199 ;      // pulse width of output signal

    while(1){;}
}
```



PRODUCING A CUSTOM WAVEFORM

- Y-axis
 - 1 step = 1 v
- X-axis
 - 1 Step = 200 us



PRACTICE PROBLEM

- Use Timer 1's output compare interrupt to toggle pin B.1 every $1000\mu\text{s}$
- Assume System clock frequency = 8 MHz. Write a program that generates waves with duty cycles of 30% and 60% on the OC1A and OC1B pins, respectively. The frequency of the generated waves should be 125 Hz



CONTROLLING A SERVO MOTOR

- We use a servo motor S3003.
- It has three wires
 - Black: Ground
 - Red: DC supply between (4.8V, 6V)
 - White: PWM signal
- The frequency of the PWM signal is 50Hz.
- This motor have a rotation range of 180°.
- To keep the motor at a given angle, we must send a PWM signal of a specific duty cycle.
- Range of duty cycle: 1% to 12%.
- See video of motor and PWM waveform.



CONTROLLING A SERVO MOTOR

Write C program that lets the user press two switches to rotate the motor left and right.

- The switches can be connected to pins of port A.
- Depending on which switch is pressed, we increment or decrement the duty cycle.
- We then produce a PWM signal on pin OC1A with
 - a period of 20000 μ s,
 - a specific duty cycle between 1% and 12%.



```

#include <avr\io.h>
int main(void) {
    unsigned int period, duty_cycle, high_time;
    unsigned char button;
    DDRA = 0b00; DDRB = 0xFF; // set port A for input, port B for output
    DDRD = 0b00100000; // set pin D.5 for output (OC1A)

    // WGM11:WGM10 = 10: with WGM13-WGM12 to select timer mode 1110
    // Fast PWM, timer 1 runs from 0 to ICR1
    // COM1A1:COM1A0 = 10: clear OC1A when compare match, set OC1A when 0
    TCCR1A = 0b10000010; // compare match occurs timer = OCR1A
    TCCR1B = 0b00011001; // WGM13:WGM12=11; CS12:CS0=001: internal clock 1MHz, no prescaler

    period = 20000; // PWM frequency = 50Hz, period = 20000us
    duty_cycle = 6; // initial duty cycle
    ICR1 = period - 1; // period of output PWM signal
    high_time = (period/100) * duty_cycle; // calculate high time
    OCR1A = high_time - 1; // set high time of output PWM signal
    while (1){
        if (button == PINA) // ignore repeated press
            continue;
        button = PINA; PORTB = button; // store button press, display on port B
        if ((button & 0b11000000) == 0b11000000) // ignore all except buttons SW6 and SW7
            continue;

        if ((button & 0b10000000) == 0) // Increment duty cycle if switch SW7 is pressed
            duty_cycle = (duty_cycle<12)?duty_cycle+1:duty_cycle;

        if ((button & 0b01000000) == 0) // Increment duty cycle if switch SW6 is pressed
            duty_cycle = (duty_cycle>1)?duty_cycle-1:duty_cycle;
        high_time = (period/100)*duty_cycle; // calculate high time
        OCR1A = high_time - 1; // set high time of output signal
    }
}

```

